Interfaces de Controladores — Módulo Comercios y Catálogo

Oct 26, 2026 · GRUPO 7

1. Objetivo y alcance

Este anexo define la interfaz de los tres controladores del módulo Comercios y Catálogo (`ComercioController`, `ProductoController`, `CatalogoController`): qué endpoints exponen, qué reciben, qué devuelven y qué errores informan. Continúa la Primera Entrega (Grupo 07) y respeta su numeración: cada endpoint, parámetro y código de error remite a un CU, RN, SRV, DTO o DM existente.

- **Dentro del alcance:** API-01 a API-05, que cubren los cinco CU core (CU-01 a CU-05).
- **Previsto, sin implementar en esta entrega:** endpoints de los CU no core (CU-06 a CU-10), en la sección 9.
- **Fuera de alcance:** autenticación (módulo Usuarios, INT-01) y notificaciones salientes (DM-05).

Donde la Primera Entrega dejaba un punto abierto (parámetros de búsqueda, origen de la identidad, formato de error), se resuelve con una decisión nueva, DM-06 a DM-10, en la sección 11. El boceto de pantallas (`PRIMER_VISTAZO_INTERFAZ`) se usa solo como referencia; ante una diferencia, prevalece la Primera Entrega.

2. Cómo interpretar un controlador

Un controlador es la puerta de entrada HTTP del módulo: recibe el pedido, lo traduce a un DTO, delega en un Service y devuelve la respuesta. No contiene reglas de negocio; esas viven en la capa Service (sección 9 de la Primera Entrega). Su **interfaz** es el contrato que ven los consumidores (Portal Web, Pedidos): ruta, verbo, entrada, salida y errores.

Recorrido de un pedido por la capa Controller:

1. Llega el pedido HTTP; Spring lo enruta al método anotado con la ruta y el verbo (`@PostMapping`, `@PatchMapping`, `@GetMapping`).
2. El cuerpo JSON se convierte en el DTO de entrada y `@Valid` aplica sus anotaciones de Bean Validation (RN-06, RN-03, RN-04). Si falla, responde 400 sin llegar al Service.
3. El controlador llama a la operación del Service (SRV-01 a SRV-05) con el DTO o los parámetros ya validados.
4. Si el Service detecta una regla incumplida, lanza una excepción del paquete `Exception`; `GlobalExceptionHandler` la traduce al código HTTP y al cuerpo `ErrorResponseDTO` (sección 5).
5. Si todo sale bien, el controlador devuelve el DTO de salida con 200 OK o 201 Created.

Cómo leer cada ficha de las secciones 6 a 8:

Parte de la ficha	Qué dice
Ruta y verbo	Dirección del endpoint y operación HTTP
Método Java	Firma del método del controlador y anotaciones

Parte de la ficha	Qué dice
Trazabilidad	CU, SRV y RN que lo justifican
Entrada	Headers, parámetros y DTO del cuerpo, con sus validaciones
Salida	DTO devuelto y código de éxito
Errores	Código HTTP, causa, excepción que lo produce y RN asociada
Ejemplo	Pedido y respuesta en JSON

3. Diagrama de clases de los controladores

Los tres controladores reciben y devuelven solo DTOs y dependen de un único Service cada uno; ninguno accede a repositorios ni a entidades.

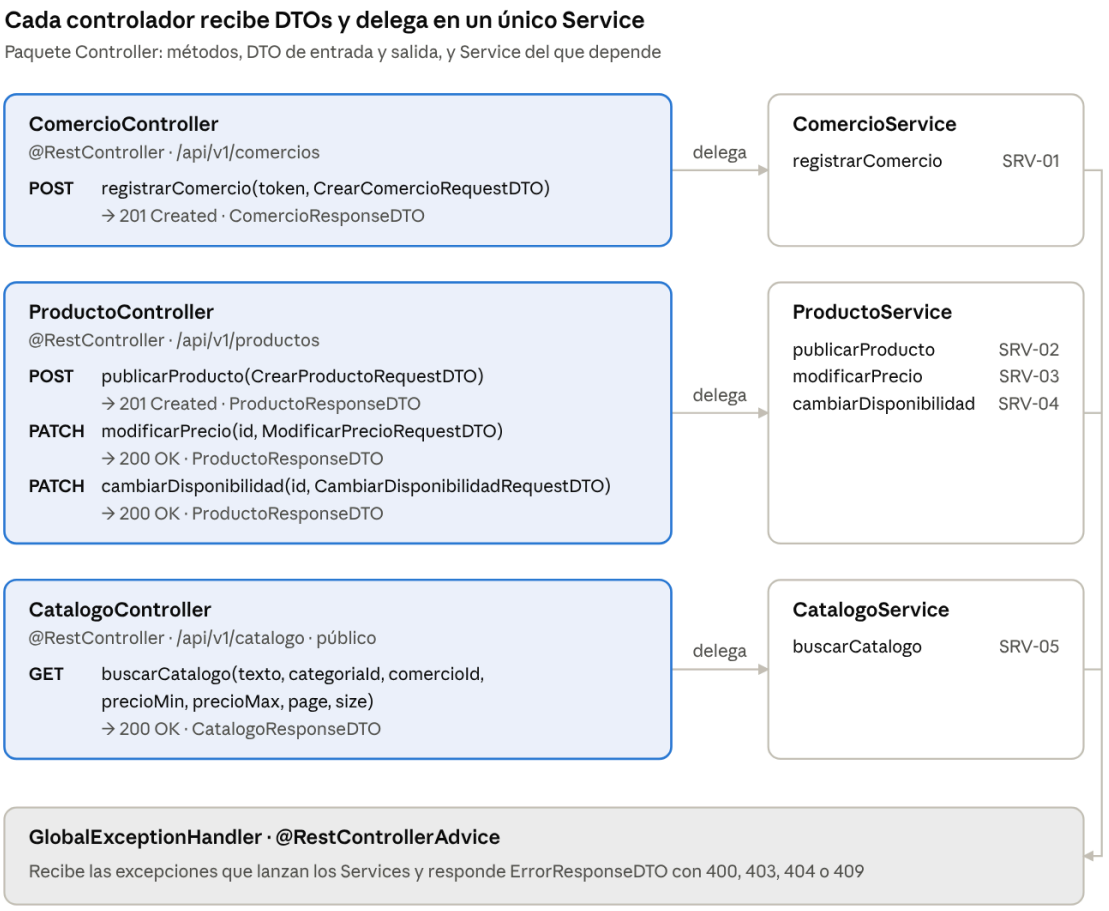
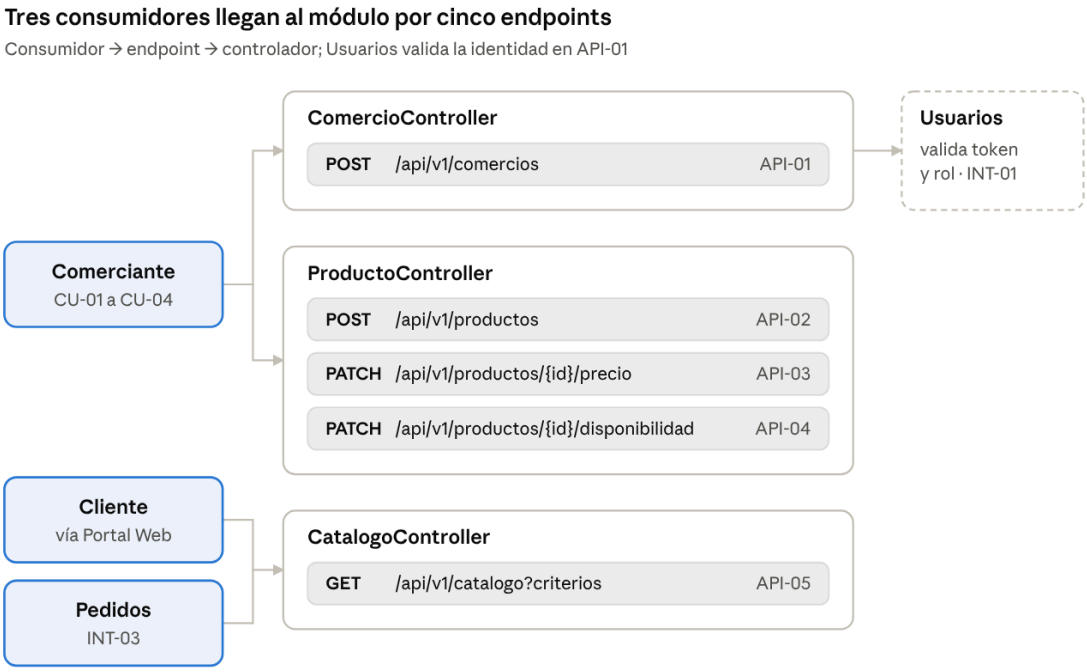


Diagrama de clases · paquete Controller

Las flechas "delega" son la única dependencia de cada controlador. Los errores no vuelven por el controlador: las excepciones de los Services las captura `GlobalExceptionHandler`, que arma el `ErrorResponseDTO`.

4. Mapa de endpoints

El Comerciante usa cuatro endpoints de escritura en dos controladores; el Cliente (vía Portal Web) y Pedidos comparten el único endpoint de lectura, API-05.



Mapa de endpoints · consumidores y controladores

Usuarios (borde punteado) es externo al módulo: solo interviene en API-01 para validar el token y el rol del comerciante (DM-06). Pedidos consume API-05 servidor a servidor con el mismo contrato que el Portal (INT-03).

5. Convenciones comunes

Todos los endpoints comparten prefijo, formato de intercambio y formato de error; las fichas siguientes solo indican lo que cambia.

- **Prefijo:** `/api/v1` (sección 12.2 de la Primera Entrega).
- **Formato:** JSON en el cuerpo, `Content-Type: application/json`. Precios como número decimal (`BigDecimal`, 2 decimales).
- **Identidad (DM-06):** el comerciante se autentica con sus credenciales en el módulo Usuarios, que le emite un token. API-01 recibe ese token en el header `Authorization: Bearer <token>` y Catálogo lo valida contra Usuarios (INT-01): identidad válida y rol Comerciante. Si no se cumple, responde 403.

- **Errores (DM-08):** toda respuesta de error usa `ErrorResponseDTO`, incluidos los 400 de validación.

Cuerpo de error:

```
{
  "timestamp": "2026-10-27T10:15:30",
  "status": 409,
  "error": "CONFLICT",
  "message": "Ya existe un comercio con el CUIT 30712345678",
  "path": "/api/v1/comercios"
}
```

Códigos HTTP y excepción que los produce:

Código	Cuándo	Excepción	Endpoints
200 OK	Operación o consulta exitosa	—	API-03, API-04, API-05
201 Created	Recurso creado	—	API-01, API-02
400 Bad Request	Falla Bean Validation del DTO, JSON mal formado o parámetro de tipo incorrecto	`MethodArgumentNotValidException`, `HttpMessageNotReadableException`, `MethodArgumentTypeMismatchException` (DM-08)	Todos
400 Bad Request	Regla de negocio sobre los datos: categoría inexistente, criterios de búsqueda inválidos	`ReglaDeNegocioException`	API-02, API-05
403 Forbidden	Usuario no válido o sin rol Comerciante (INT-01)	`AccesoNoAutorizadoException`	API-01
404 Not Found	Producto inexistente; comercio inexistente o inactivo al publicar (DM-09)	`RecursoNoEncontradoException`	API-02, API-03, API-04
409 Conflict	CUIT ya registrado (RN-01)	`RecursoDuplicadoException`	API-01
409 Conflict	Comercio del producto inactivo (RN-02)	`ComercioInactivoException`	API-03, API-04

Los mensajes de error de los ejemplos son ilustrativos; el texto final lo define cada Service al lanzar la excepción.

6. ComercioController

`ComercioController` expone un endpoint en esta entrega: el alta de comercio (API-01). Ruta base: `@RequestMapping("/api/v1/comercios")`. Depende de `ComercioService`.

API-01 · Registrar comercio

Campo	Valor
Ruta y verbo	`POST /api/v1/comercios`
Método Java	`ResponseEntity<ComercioResponseDTO> registrarComercio(@RequestHeader("Authorization") String token, @Valid @RequestBody CrearComercioRequestDTO request)`
Trazabilidad	CU-01 · SRV-01 `registrarComercio()` · RN-01, RN-06 · INT-01 · SEC-01
Entrada	Header `Authorization` (DM-06) + DTO-01 `CrearComercioRequestDTO`
Salida	DTO-02 `ComercioResponseDTO` · 201 Created, con `estado = "ACTIVO"`

Validaciones del cuerpo (DTO-01):

Campo	Tipo	Regla	RN
`cuit`	String	Obligatorio; exactamente 11 dígitos, sin guiones	RN-06
`razonSocial`	String	Obligatorio, no vacío	RN-06
`direccion`	String	Obligatorio, no vacío	RN-06
`rubro`	String	Obligatorio, no vacío	RN-06

Errores:

Código	Causa	Excepción	RN / INT
400	Campo faltante, vacío o CUIT con formato inválido	`MethodArgumentNotValidException`	RN-06
403	Token inválido o usuario sin rol Comerciante	`AccesoNoAutorizadoException`	INT-01
409	CUIT ya registrado (`findByCuit`, REP-01)	`RecursoDuplicadoException`	RN-01

Ejemplo:

```
POST /api/v1/comercios
Authorization: Bearer eyJhbGciOi...

{
  "cuit": "30712345678",
  "razonSocial": "La Esquina SRL",
  "direccion": "Av. Corrientes 1450, CABA",
  "rubro": "Hamburguesería"
```

```
}

201 Created
{
  "id": 1,
  "cuit": "30712345678",
  "razonSocial": "La Esquina SRL",
  "estado": "ACTIVO"
}
```

7. ProductoController

`ProductoController` expone tres endpoints: alta de producto (API-02), modificación de precio (API-03) y cambio de disponibilidad (API-04). Los tres devuelven el mismo DTO-04 `ProductoResponseDTO` (DM-01). Ruta base: `@RequestMapping("/api/v1/productos")`. Depende de `ProductoService`.

API-02 · Publicar producto

Campo	Valor
Ruta y verbo	`POST /api/v1/productos`
Método Java	`ResponseEntity<ProductoResponseDTO> publicarProducto(@Valid @RequestBody CrearProductoRequestDTO request)`
Trazabilidad	CU-02 · SRV-02 `publicarProducto()` · RN-02, RN-03, RN-04, RN-06 · SEC-02
Entrada	DTO-03 `CrearProductoRequestDTO`
Salida	DTO-04 `ProductoResponseDTO` · 201 Created, con `disponible = true`

Validaciones del cuerpo (DTO-03):

Campo	Tipo	Regla	RN
`nombre`	String	Obligatorio, no vacío	RN-06
`descripcion`	String	Obligatorio, no vacío	RN-06
`precio`	BigDecimal	Obligatorio, mayor a cero	RN-04
`categoriaIds`	List<Long>	Al menos un elemento, sin nulos	RN-03
`comercioId`	Long	Obligatorio	RN-06

Errores:

Código	Causa	Excepción	RN
400	Campo faltante, precio ≤ 0 o lista de categorías vacía	`MethodArgumentNotValidException`	RN-06, RN-04, RN-03

Código	Causa	Excepción	RN
400	Alguna categoría informada no existe (`findAllById`, REP-06)	`ReglaDeNegocioException`	RN-03
404	Comercio inexistente o inactivo (`findByIdAndEstado`, REP-02; DM-09)	`RecursoNoEncontradoException`	RN-02

Ejemplo:

```
POST /api/v1/productos

{
  "nombre": "Hamburguesa clásica",
  "descripcion": "Carne, queso, lechuga y tomate.",
  "precio": 8500.00,
  "categoriaIds": [1],
  "comercioId": 1
}

201 Created
{
  "id": 10,
  "nombre": "Hamburguesa clásica",
  "precio": 8500.00,
  "disponible": true,
  "comercioId": 1,
  "categorias": [{ "id": 1, "nombre": "Hamburguesas" }]
}
```

API-03 · Modificar precio

Campo	Valor
Ruta y verbo	`PATCH /api/v1/productos/{id}/precio`
Método Java	`ResponseEntity<ProductoResponseDTO> modificarPrecio(@PathVariable Long id, @Valid @RequestBody ModificarPrecioRequestDTO request)`
Trazabilidad	CU-03 · SRV-03 `modificarPrecio(productoId, nuevoPrecio)` · RN-02, RN-04 · SEC-03
Entrada	Path `id` (Long, id del producto) + DTO-05 `ModificarPrecioRequestDTO`
Salida	DTO-04 `ProductoResponseDTO` · 200 OK

Validaciones del cuerpo (DTO-05): `nuevoPrecio` (BigDecimal) es obligatorio y mayor a cero (RN-04).

Errores:

Código	Causa	Excepción	RN
400	`nuevoPrecio` faltante o ≤ 0; `id` no numérico	`MethodArgumentNotValidException`, `MethodArgumentTypeMismatchException`	RN-04
404	Producto inexistente (`findById`, REP-03)	`RecursoNoEncontradoException`	—
409	Comercio del producto inactivo (REP-02)	`ComercioInactivoException`	RN-02

Ejemplo:

```
PATCH /api/v1/productos/10/precio

{ "nuevoPrecio": 9200.00 }

200 OK
{
  "id": 10,
  "nombre": "Hamburguesa clásica",
  "precio": 9200.00,
  "disponible": true,
  "comercioId": 1,
  "categorias": [{ "id": 1, "nombre": "Hamburguesas" }]
}
```

API-04 · Cambiar disponibilidad

Campo	Valor
Ruta y verbo	`PATCH /api/v1/productos/{id}/disponibilidad`
Método Java	`ResponseEntity<ProductoResponseDTO> cambiarDisponibilidad(@PathVariable Long id, @Valid @RequestBody CambiarDisponibilidadRequestDTO request)`
Trazabilidad	CU-04 · SRV-04 `cambiarDisponibilidad(productoId, nuevoEstado)` · RN-02 · SEC-04
Entrada	Path `id` (Long, id del producto) + DTO-06 `CambiarDisponibilidadRequestDTO`
Salida	DTO-04 `ProductoResponseDTO` · 200 OK

Validaciones del cuerpo (DTO-06): `disponible` (Boolean) es obligatorio. Se usa `Boolean` y no `boolean` para que un cuerpo sin el campo sea rechazado y no quede en `false` (DM-10).

Errores:

Código	Causa	Excepción	RN
400	`disponible` faltante; `id` no numérico (DM-10)	`MethodArgumentNotValidException`, `MethodArgumentTypeMismatchException`	RN-06
404	Producto inexistente (REP-03)	`RecursoNoEncontradoException`	—
409	Comercio del producto inactivo (REP-02)	`ComercioInactivoException`	RN-02

Ejemplo:

```
PATCH /api/v1/productos/10/disponibilidad

{ "disponible": false }

200 OK
{
  "id": 10,
  "nombre": "Hamburguesa clásica",
  "precio": 9200.00,
  "disponible": false,
  "comercioId": 1,
  "categorias": [{ "id": 1, "nombre": "Hamburguesas" }]
}
```

El cambio impacta en API-05: un producto con `disponible = false` deja de aparecer en el catálogo (RN-05).

8. CatalogoController

`CatalogoController` expone la búsqueda pública del catálogo (API-05). Es el endpoint de lectura que consumen el Portal Web y Pedidos (INT-03) y no requiere autenticación. Ruta base: `@RequestMapping("/api/v1/catalogo")`. Depende de `CatalogoService`.

API-05 · Buscar catálogo

Campo	Valor
Ruta y verbo	`GET /api/v1/catalogo`
Método Java	`ResponseEntity<CatalogoResponseDTO> buscarCatalogo(@RequestParam(required = false) String texto, @RequestParam(required = false) List<Long> categoriaId, @RequestParam(required = false) Long comercioId, @RequestParam(required = false) BigDecimal precioMin, @RequestParam(required = false) BigDecimal precioMax, @RequestParam(defaultValue = "0") int page, @RequestParam(defaultValue = "20") int size)`
Trazabilidad	CU-05 · SRV-05 `buscarCatalogo(criterios)` · RN-05 · REP-04 · INT-03 · SEC-05

Campo	Valor
Entrada	Parámetros de consulta (DM-07), todos opcionales
Salida	DTO-07 `CatalogoResponseDTO` · 200 OK, también con lista vacía

Parámetros de consulta (DM-07):

Parámetro	Tipo	Filtra por	Regla
`texto`	String	Nombre o descripción del producto, o razón social del comercio (sin distinguir mayúsculas)	Opcional
`categoriaId`	Long, repetible	Productos con al menos una de las categorías (`?categoriaId=1&categoriaId=3`)	Opcional
`comercioId`	Long	Productos de un comercio	Opcional
`precioMin`	BigDecimal	Precio ≥ valor	Opcional; ≥ 0
`precioMax`	BigDecimal	Precio ≤ valor	Opcional; ≥ 0 y ≥ `precioMin`
`page`	int	Número de página, desde 0	Por defecto 0; ≥ 0
`size`	int	Tamaño de página	Por defecto 20; entre 1 y 100

Sin importar los filtros, la consulta solo devuelve productos con `disponible = true` cuyo comercio tiene `estado = "ACTIVO"` (RN-05).

Errores:

Código	Causa	Excepción	RN
400	`precioMin` > `precioMax`, valores negativos, `page` < 0 o `size` fuera de 1 a 100	`ReglaDeNegocioException`	—
400	Parámetro con tipo inválido (por ejemplo `comercioId=abc`)	`MethodArgumentTypeMismatchException`	—

Sin resultados no es un error: devuelve 200 con `items` vacío y `total = 0`.

Ejemplo:

```
GET /api/v1/catalogo?texto=hamburguesa&categoriaId=1&precioMax=9000&page=0&size=12

200 OK
{
  "items": [
    {
```

```
      "id": 10,
      "nombre": "Hamburguesa clásica",
      "precio": 8500.00,
      "disponible": true,
      "comercioId": 1,
      "categorias": [{ "id": 1, "nombre": "Hamburguesas" }]
    }
  ],
  "page": 0,
  "size": 12,
  "total": 1
}
```

9. Endpoints previstos para CU no core

Los CU-06 a CU-10 no se implementan en esta entrega, pero se reserva su interfaz para que la API crezca sin romper el contrato de API-01 a API-05. Se numeran API-06 en adelante; aplican las mismas convenciones de la sección 5.

ID	Verbo y ruta	Controlador	CU	Entrada → Salida	Errores	Nota
API-06	`GET /api/v1/productos/{id}`	`ProductoController`	CU-06	path `id` → DTO-04	404	Ruta definida en la Primera Entrega. Solo productos disponibles de comercios activos (RN-05)
API-07	`GET /api/v1/comercios/{comercioId}/sucursales`	`SucursalController`	CU-07	path → `List<SucursalDTO>`	404	Sucursales de un comercio (REL-01)
API-08	`POST /api/v1/comercios/{comercioId}/sucursales`	`SucursalController`	CU-07	`SucursalDTO` → `SucursalDTO`	400, 404, 409	409 si el comercio está inactivo (RN-02)
API-09	`PUT` / `DELETE /api/v1/comercios/{comercioId}/sucursales/{id}`	`SucursalController`	CU-07	`SucursalDTO` → `SucursalDTO` / 204	400, 404, 409	Ídem RN-02
API-10	`GET /api/v1/categorias`	`CategoriaController`	CU-08	— → `List<CategoriaDTO>`	—	Lectura pública del catálogo global (DM-02)
API-11	`POST` / `PUT` / `DELETE`	`CategoriaController`	CU-08	`CategoriaDTO` →	400, 403, 404	Solo rol Administrador,

ID	Verbo y ruta	Controlador	CU	Entrada → Salida	Errores	Nota
	`/api/v1/categorias[/{id}]`			`CategoriaDTO` / 204		validado vía INT-01 (DM-06)
API-12	PATCH `/api/v1/productos/{id}/estado`	`ProductoController`	CU-09	`{ "activo": boolean }` → DTO-04	400, 404, 409	Soft-delete. Requiere agregar un atributo de estado a ENT-02, hoy inexistente
API-13	PUT `/api/v1/comercios/{id}`	`ComercioController`	CU-10	razón social, dirección, rubro → DTO-02	400, 403, 404	El CUIT no se acepta en el cuerpo: es inmutable (RN-01)

`SucursalDTO` y `CategoriaDTO` ya existen en el código. Los DTO de entrada de API-12 y API-13 se definen cuando se implementen esos CU.

10. De la pantalla al controlador

Cada acción del boceto de interfaz dispara un endpoint concreto; esta tabla sirve para leer los controladores desde el lado del usuario. El boceto es preliminar y, donde difiere de la Primera Entrega, manda la Primera Entrega.

Pantalla del boceto	Acción del usuario	Endpoint	Controlador
01 Catálogo (Cliente)	Filtrar por categoría o comercio, paginar	API-05 `GET /catalogo`	`CatalogoController`
02 Detalle de comercio (Cliente)	Ver productos del comercio	API-05 `GET /catalogo?comercioId=`	`CatalogoController`
02 Detalle de comercio (Cliente)	Ver sucursales	API-07 (previsto)	`SucursalController`
03 Mis productos (Comercio)	Activar o desactivar "Disponible"	API-04 `PATCH /productos/{id}/disponibilidad`	`ProductoController`
04 Nuevo producto (Comercio)	Cargar las categorías para elegir	API-10 (previsto)	`CategoriaController`
04 Nuevo producto (Comercio)	Crear producto	API-02 `POST /productos`	`ProductoController`

Pantalla del boceto	Acción del usuario	Endpoint	Controlador
05 Modificar precio (Comercio)	Guardar el nuevo precio	API-03 `PATCH /productos/{id}/precio`	`ProductoController`
06 Alta de comercio	Dar de alta el comercio	API-01 `POST /comercios`	`ComercioController`
07 Estados del sistema	Mostrar 400, 403, 404, 409	`ErrorResponseDTO.message`	`GlobalExceptionHandler`

Diferencias del boceto que no se adoptan, por respeto al contrato de la Primera Entrega:

- El catálogo del boceto muestra productos "No disponible"; API-05 los excluye (RN-05).
- "Mis productos" lista también los no disponibles; API-05 no sirve para esa vista. Queda pendiente un listado privado del comerciante, a definir junto con CU-09.
- El formulario de producto incluye un selector "Disponible"; DTO-03 no lo tiene y el alta siempre crea con `disponible = true` (CU-02).
- El alta de comercio aparece hecha por un Administrador; el contrato la asigna al Comerciante autenticado (CU-01, DM-06).
- Las tarjetas muestran descripción y razón social; DTO-04 no las incluye y entrega `comercioId`.

11. Nuevas decisiones de modelado

Cinco decisiones completan los puntos que la Primera Entrega dejaba abiertos para definir la interfaz. Continúan la numeración de la sección 7.

ID	Decisión	Alternativas consideradas	Motivo	Impacta en
DM-06	El comerciante se autentica con sus credenciales en Usuarios, que emite un token. API-01 lo recibe en `Authorization: Bearer <token>` y Catálogo valida identidad y rol Comerciante contra Usuarios.	`usuarioId` dentro de DTO-01; header propio `X-Usuario-Id`.	La autenticación pertenece a Usuarios (fuera de alcance). Un id en el cuerpo podría falsificarse; el token lo verifica quien lo emitió. Cumple el paso 3 de CU-01 y su 403.	API-01, SRV-01, INT-01, CU-01
DM-07	API-05 recibe `texto`, `categoriaId` (repetible), `comercioId`,	Un único parámetro `q` libre; cuerpo JSON en un `POST`	CU-05 pide filtrar por texto, categoría, comercio y rango de precio. Parámetros con nombre en un GET son cacheables y	API-05, SRV-05, REP-04

ID	Decisión	Alternativas consideradas	Motivo	Impacta en
	`precioMin`, `precioMax`, `page` y `size`.	`/catalogo/busqueda`.	fáciles de consumir para Portal y Pedidos (INT-03). `texto` también busca por razón social, porque CU-05 incluye consultar comercios.	
DM-08	Todo error responde con `ErrorResponseDTO`, incluidos los 400 de Bean Validation, JSON mal formado y tipo de parámetro inválido.	Dejar la respuesta por defecto de Spring para la validación.	Un único formato de error simplifica a los consumidores. Requiere agregar en `GlobalExceptionHandler` los handlers de `MethodArgumentNotValidException`, `HttpMessageNotReadableException` y `MethodArgumentTypeMismatchException`.	Todos los endpoints, RN-06
DM-09	Al publicar (API-02), comercio inexistente e inactivo responden igual: 404. Al modificar (API-03, API-04), comercio inactivo responde 409.	409 también en API-02, con una consulta extra para distinguir.	En API-02 `findByIdAndEstado` (REP-02) no distingue entre inexistente e inactivo, y CU-02 fija 404. En API-03/04 el producto ya existe, así que su comercio existe: si no está activo es un conflicto de estado (CU-03, CU-04).	API-02, API-03, API-04, RN-02
DM-10	API-04 agrega 400 a sus errores: `disponible` es `Boolean` obligatorio.	`boolean` primitivo, que toma `false` si falta el campo.	Un cuerpo vacío no debe deshabilitar un producto sin querer. La tabla de la sección 12.2 de la Primera Entrega listaba solo 404 y 409.	API-04, DTO-06

12. Matriz de trazabilidad de controladores

Cada método de controlador se justifica por un CU y delega en exactamente una operación de servicio; ninguno queda sin trazabilidad.

Controlador	Método	API	CU	SRV	Entrada	Salida	RN	INT / DM
`ComercioController`	`registrarComercio`	API-01	CU-01	SRV-01	DTO-01 + token	DTO-02	RN-01, RN-06	INT-01, DM-06, DM-08
`ProductoController`	`publicarProducto`	API-02	CU-02	SRV-02	DTO-03	DTO-04	RN-02, RN-03, RN-04, RN-06	DM-01, DM-08, DM-09
`ProductoController`	`modificarPrecio`	API-03	CU-03	SRV-03	`id` + DTO-05	DTO-04	RN-02, RN-04	DM-01, DM-08, DM-09
`ProductoController`	`cambiarDisponibilidad`	API-04	CU-04	SRV-04	`id` + DTO-06	DTO-04	RN-02	DM-01, DM-09, DM-10
`CatalogoController`	`buscarCatalogo`	API-05	CU-05	SRV-05	parámetros de consulta	DTO-07	RN-05	INT-03, DM-07, DM-08